

Modern Express + TypeScript Setup Guide

Step-by-step production blueprint to configure Node.js, Express, TypeScript 5+, tsx, and Nodemon.

1. Technology Stack Overview

Modern Node.js (v20, v22, v24+) introduced strict ES module resolution where legacy ts-node fails with Cannot read properties of undefined (reading 'fileExists'). We use **tsx** (esbuild-powered) for instant hot-reload and **tsc** for production compilation.

Component	Package / Version	Purpose
Runtime	Node.js (v20+)	Modern JavaScript runtime engine
HTTP Framework	express (^5.2.x)	REST API routing & middleware pipeline
Language & Types	typescript (^5.8.x)	Static typing & production build compiler
Dev Runner	tsx (^4.23.x)	Instant TypeScript execution (zero-config esbuild)
File Watcher	nodemon (^3.1.x)	Watches src/ and reloads tsx on change
Typings	@types/node, @types/express, @types/cors	Compile-time type declarations and IntelliSense

2. Project Directory Structure

server/	
node_modules/	
src/	
index.ts	# Express application entry point
nodemon.json	# Live-reload watcher configuration
package.json	# Dependencies and NPM scripts
tsconfig.json	# TypeScript compiler configuration

3. Installation Commands

1. Initialize project
npm init -y
2. Install production dependencies
npm install express
3. Install TypeScript, tsx, nodemon, and type bindings
npm install -D typescript tsx nodemon @types/node @types/express @types/cors

KEY INSIGHT: Why tsx over ts-node? tsx runs TypeScript directly without ahead-of-time type checks or broken loaders, starting up in <50ms while remaining fully compatible with modern Node.js releases.

4. TypeScript Configuration (tsconfig.json)

{
"compilerOptions": {
"target": "ES2022",
"module": "CommonJS",
"moduleResolution": "node",
"outDir": "./dist",
"rootDir": "./src",
"strict": true,
"esModuleInterop": true,
"skipLibCheck": true,
"forceConsistentCasingInFileNames": true
},
"include": ["src/**/*"]
}

5. Nodemon Configuration (nodemon.json)

Configure nodemon to watch the src directory and execute TypeScript files via tsx:

```
{
  "watch": ["src"],
  "ext": "ts",
  "exec": "tsx src/index.ts"
}
```

6. Server Entry Point (src/index.ts)

```
import express, { Request, Response } from "express";

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

app.get("/", (req: Request, res: Response) => {
  res.send("SmartResidence API is running");
});

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

7. NPM Scripts (package.json)

```
"scripts": {
  "dev": "nodemon",
  "dev:tsx": "tsx watch src/index.ts",
  "build": "tsc",
  "start": "node dist/index.js"
}
```

8. CLI Workflow Commands

- **Development Mode (Live Reloading):** npm run dev
- **Production TypeScript Build:** npm run build (compiles TS files to dist/)
- **Production Runtime:** npm start

9. Troubleshooting Common Pitfalls

Error	Cause & Fix
TypeError: Cannot read properties of undefined (reading 'fileExists')	ts-node failed to bind to TypeScript compiler. Fix: Install typescript and set exec: 'tsx src/index.ts' in nodemon.json.
Cannot find module 'express' or its type declarations	Missing TypeScript typings. Fix: Run npm i -D @types/express @types/node.
EADDRINUSE: address already in use :::3000	Port is occupied by another process. Fix: Terminate the active task or run PORT=5000 npm run dev.